# ATIVIDADE 4
## BUSCA HEURÍSTICA

Lucas Gonçalves Balduino - 22409139

INTELIGENCIA ARTIFICIAL

# Introdução

A busca heurística é uma técnica de resolução de problemas, ela se baseia na função heurística para determinar a partir de parâmetros (Valores Heurísticos), qual o melhor passo para se dar em busca de um objetivo. Isso permite, encontrar um caminho mais rápido e de forma mais eficiente. Essa técnica é utilizada em diversos campos, aplicativos de navegação, optimização e estratégias de jogo.

# Algoritmo de Djkstra

O Algoritmo de Dijkstra pertence a uma outra categoria de busca, chamada de busca cega, ele se encaixa nessa categoria justamente por não utilizar de heurísticas para realizar a busca, ele tenta cada caminho até encontrar o caminho ideal para o objetivo. Dessa forma, o algoritmo não é considerado eficiente por demandar mais tempo e poder computacional em relação à outras técnicas. Ele pode ser identificado em roteamento de redes (pacotes de internet), por não possuir um caminho físico exato, e tentar todas as portas até achar o caminho.

# A* (A Star)

O Algoritmo A* é considerado o estado da arte, ele junta o melhor das duas técnicas para encontrar o melhor caminho de forma eficiente. Dessa forma, esse algoritmo utiliza de heurísticas para basear a tomada de decisão mas também explora outros caminhos para garantir que é o caminho ideal, utilizando uma funcao f(n) = g(n) + h(n), sendo g(n) a busca cega (custo real) do algoritmo de Dijkstra e o h(n) a heurística utilizada na busca Gulosa. Seu uso está presente, em grande parte, na tomada de decisão de IA's em jogos e navegação.

# A* (A Star)

```
import heapq # Biblioteca para Fila de Prioridade
```

```
# 1. Definição da Função Heurística h(n)
# Usaremos a Distância de Manhattan, ideal para grades onde não há movimento diagonal.
# Ela calcula a soma das diferenças absolutas das coordenadas x e y.
def heuristic(a, b):
    (x1, y1) = a
    (x2, y2) = b
    return abs(x1 - x2) + abs(y1 - y2)
.")
```

# A* (A Star)

```python
# 2. O Algoritmo A*
def a_star_search(graph, start, goal):
    """
    graph: Matriz (grade) onde 0 é livre e 1 é obstáculo.
    start: Tupla (x, y) de início.
    goal: Tupla (x, y) do objetivo.
    """

    rows, cols = len(graph), len(graph[0])

    # fila_prioridade armazena (custo_f, (x, y))
    # Iniciamos apenas com o ponto de partida, onde f = 0.
    fila_prioridade = [(0, start)]
    heapq.heapify(fila_prioridade) # Transforma em uma heap (fila de prioridade)
```

# A* (A Star)

```
# Dicionários para rastrear o caminho e o custo g
came_from = {start: None} # Quem me levou a esse nó? (Para reconstruir o caminho no fim)
cost_so_far = {start: 0}   # custo g(n): Custo real do início até aqui.

# Conjunto de nós já visitados e fechados
closed_set = set()

while fila_prioridade:
    # Pega o nó com o MENOR custo 'f' da fila
    f_score_atual, current = heapq.heappop(fila_prioridade)
```

# A* (A Star)

```
# Se chegamos ao objetivo, sucesso!
if current == goal:
    break

# Evita reprocessar nós já visitados
if current in closed_set:
    continue
closed_set.add(current)

x, y = current
# Lista de possíveis vizinhos (cima, baixo, esquerda, direita)
for dx, dy in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
    next_node = (x + dx, y + dy)
    nx, ny = next_node

    # Verificação de limites da grade e obstáculos (1 = muro)
    if 0 <= nx < rows and 0 <= ny < cols and graph[nx][ny] == 0:

        # Novo custo g(n) = custo até o nó atual + custo da aresta (neste caso, 1)
        new_cost = cost_so_far[current] + 1
```

# A* (A Star)

```
# Se ainda não visitamos esse vizinho OU encontramos um caminho mais barato até ele
                if next_node not in cost_so_far or new_cost < cost_so_far[next_node]:
                    cost_so_far[next_node] = new_cost
                    # Cálculo de f(n) = g(n) + h(n)
                    f_score = new_cost + heuristic(next_node, goal)

                    # Adiciona à fila de prioridade
                    heapq.heappush(fila_prioridade, (f_score, next_node))
                    # Registra a origem para reconstrução do caminho
                    came_from[next_node] = current

    # Reconstrução do caminho final, do fim para o início
    if goal not in came_from:
        return None # Caminho não encontrado

    path = []
    current_node = goal
    while current_node is not None:
        path.append(current_node)
        current_node = came_from[current_node]
    path.reverse() # Inverte para ter a ordem Início -> Fim
    return path
```

# A* (A Star)

```
# --- Exemplo de Uso Prático ---

# 3. Criação do Mapa (0=livre, 1=obstáculo)
# Temos um muro em 'shape de U' que o Dijkstra e a Busca Gulosa teriam dificuldade
grade_exemplo = [
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0], # Muro
    [0, 0, 0, 1, 0], # Muro
    [0, 1, 1, 1, 0], # Muro
    [0, 0, 0, 0, 0]
]

inicio = (0, 0) # Ponto Superior Esquerdo
fim = (4, 4)    # Ponto Inferior Direito
```

# A* (A Star)

```python
# 5. Visualização do Resultado
if resultado:
    print(f"Caminho encontrado com {len(resultado) - 1} passos:")
    # Imprime a grade com o caminho marcado
    for r in range(len(grade_exemplo)):
        linha = ""
        for c in range(len(grade_exemplo[0])):
            if (r, c) == inicio: linha += " S " # Start
            elif (r, c) == fim: linha += " E "  # End
            elif (r, c) in resultado: linha += " . " # Path
            elif grade_exemplo[r][c] == 1: linha += "###" # Obstacle
            else: linha += " - " # Empty
        print(linha)
else:
    print("Infelizmente, não existe caminho possível para este destino.")
```

# Estado da Arte

link para o artigo

Este artigo ilustra o uso de algoritmos de busca heurística para a optmização de prompts de LLM (Large Language Models), fazendo o uso de heurísticas para sistematicamente descobrir e refinar prompts.

# Referências


- Artigo GeeksforGeeks (Busca Heurística) GEEKSFORGEEKS. Heuristic Search Techniques in AI. 2023. Disponível em: link. Acesso em: 26 mar. 2026.
- Artigo Medium (Gabriel Luizone) LUIZONE, Gabriel. Descobrindo a eficácia da busca heurística. Medium, 2023. Disponível em: link. Acesso em: 26 mar. 2026.
- Aula PUCPR (Prof. Fabricio Enembreck) ENEMBRECK, Fabricio. IA - Aula 2: Busca Heurística. Curitiba: PPGIA/PUCPR. Disponível em: link. Acesso em: 26 mar. 2026.
- Tutorial W3Schools (Algoritmo de Dijkstra) W3SCHOOLS. DSA Dijkstra's Algorithm. Tradução de Google Translate. Disponível em: link. Acesso em: 26 mar. 2026.
- Artigo GeeksforGeeks (Algoritmo de Dijkstra) GEEKSFORGEEKS. Dijkstra's Shortest Path Algorithm: Greedy Algo-7. 2024. Disponível em: link. Acesso em: 26 mar. 2026.
- Tutorial DataCamp (Algoritmo A)* DATACAMP. O que é o algoritmo A?* Tutorial de IA. Disponível em: link. Acesso em: 26 mar. 2026.

# OBRIGADO

For your attention